

Generative Model Based Standard Cell Timing Library Characterization

Hao-Yu Wu¹, Hsin-Tzu Chang¹, Shiuan-Yun Ding¹, Iris Hui-Ru Jiang¹

Benson Tsao², Vinson Wu², Wei-Kai Shih²

¹Graduate Institute of Electronics Engineering, National Taiwan University, Taipei, Taiwan

²Synopsys, Inc., Hsinchu, Taiwan

email: {kobe7682477, sheena9101019, mirkat.ding, huiru.jiang}@gmail.com; {ytsao, vinsonwu, Wei-Kai.Shih}@synopsys.com

Abstract—Accurate cell timing characterization is essential, on which static timing analysis relies to verify timing performance and ensure design robustness across various PVT conditions (corners). The corner explosion in modern design amplifies the efficiency and scalability challenge for accurate characterization. However, the conventional characterization approach of SPICE simulation alone becomes prohibitively expensive due to the increasing computational complexity and the amount of characterized data. In this paper, we view the characterization problem from a generative modeling perspective to tackle the efficiency and scalability challenge. With a hybrid of generative adversarial network (GAN) and autoencoder, our generative model learns and generalizes among various timing arcs and corners. Experimental results demonstrate that the proposed framework achieves high accuracy and extensibility while reducing the runtime significantly.

Index Terms—Library Characterization, Corner Explosion, Machine Learning, Generative Model, Corner Selection

I. INTRODUCTION

Standard cell timing library characterization involves analyzing the timing behavior of each standard cell and recording it in the form of look-up tables (LUTs) indexed by input transition and output capacitance, as shown in Table I. Cell timing library provides key timing data, such as cell propagation delays and transition times, etc. The characterized data are utilized by Static Timing Analysis (STA) to evaluate timing paths and ensure design functions properly across different process, voltage, and temperature (PVT) conditions, so-called corners. As fine-grained PVT resolution becomes increasingly important for analyzing design robustness, the need to characterize more corners has grown, adding complexity to the characterization process.

SPICE simulation is typically used to characterize timing libraries, as it provides accurate timing values through extensive computations. However, it faces two main issues. First, its high computational complexity results in long runtime, creating challenges for efficiency and scalability, particularly concerning corner explosion [1]–[3], which poses a significant risk to the industry. Second, SPICE simulation requires input files and configurations from the foundry, as illustrated in Fig. 1. If the foundry needs to maintain data confidentiality, access to the necessary locked data marked in Fig. 1 becomes unavailable. This loss of access causes considerable inconvenience. There-

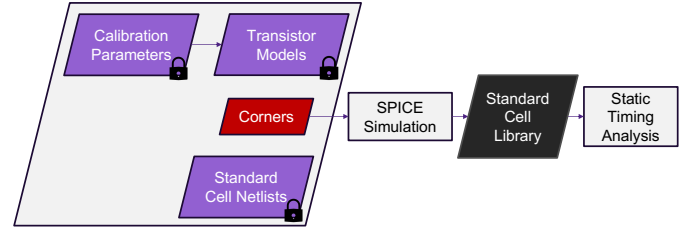


Fig. 1: Conventional library characterization.

fore, it is crucial to develop a fast characterization approach that does not depend on locked input data.

Previous research has explored various aspects of library generation. For example, cell layout generation techniques such as the use of reinforcement learning (RL) for design rule checking (DRC) [4] and the automatic cell layout generation (ACLG) framework proposed by [5], primarily focus on optimizing placement and routing processes. Additionally, [6] presents the first drain-to-drain abutment (DDA)-aware placement algorithm. While these contributions are significant, they highlight a broader need for improved methodologies in timing library characterization.

In the realm of timing library characterization, graph learning has emerged as a prominent technique by representing standard cell netlists as graphs. This approach is particularly valuable when graphical information about the cell netlist is given. For example, [7] leverages a Graph Neural Network (GNN) to capture cell structure information and interpolate fine-grained driving strength into the library. In contrast, [8] employs heterogeneous graph learning to account for parasitic RC. However, these graph learning methods require access to cell structure information, which may be confidential in certain situations as marked in Fig. 1.

Machine learning (ML) techniques have also been utilized specifically for timing library characterization, focusing on critical cell features to enhance accuracy. For instance, [9] focuses on variation modeling using a Feedforward Neural Network (FFN). It highlights the trade-off between the number of SPICE-simulated corners required and prediction accuracy, which is an important consideration when evaluating library characterization methods. Additionally, [10] proposes Aadam, which integrates their FFN models into the STA engine for aging library characterization. ML-based methods have proven effective for library characterization. However, most models

This work was supported in part by Synopsys, Inc. and the National Science and Technology Council, Taiwan.

TABLE I: A sample cell timing look-up table. Red numbers indicate the pivot value associated with the pivot input transition (ns) and pivot output capacitance (ff).

		Output Capacitance							
		0.15	0.34	1.02	2.31	4.31	7.13	10.83	
Input Transition	0.003	0.01199	0.01374	0.01951	0.03052	0.04761	0.07138	0.10250	
	0.005	0.01350	0.01527	0.02106	0.03207	0.04916	0.07292	0.10410	
	0.014	0.01898	0.02073	0.02651	0.03745	0.05439	0.07805	0.10940	
	0.029	0.02834	0.03015	0.03602	0.04698	0.06395	0.08783	0.11920	
	0.054	0.03960	0.04161	0.04764	0.05858	0.07551	0.09936	0.13080	
	0.088	0.05274	0.05492	0.06095	0.07203	0.08901	0.11270	0.14410	
	0.133	0.06691	0.06935	0.07601	0.08701	0.10390	0.12770	0.15890	

proposed in previous work can only handle specific cells or timing arcs, which necessitates the development of numerous models to fully characterize the library. On the other hand, FFN models often struggle with over-fitting, particularly when training data is limited.

The challenges mentioned above motivate us to view the timing characterization problem from a new angle to enhance runtime, accuracy, and generality. We can imagine that each timing look-up table is a point lying on a high-dimensional space. By traversing on the space along the direction of corner changing, we can reveal its counterparts under different corners. Therefore, from a generative modeling perspective, we choose a generative adversarial network (GAN) as the backbone. For generating accurate timing tables, we further integrate an autoencoder to remove the randomness from GAN. Unlike FFN models, which rely solely on loss backtracking, generative models seek additional insights from training data for better prediction performance.

To the best of our knowledge, we are the first to design a generative model for cell timing library characterization. Our key contributions are as follows:

- We introduce a generative model that enables fast and accurate timing library generation without requiring confidential data from the foundry.
- Our generative model generalizes the characterization of timing values across various cells and timing arcs, eliminating the need for separate models for each individual cell or timing arc.
- Our framework effectively reduces the volume of SPICE-simulated data and overall runtime and possesses extensibility for tackling corner explosion.

The remainder of this paper is organized as follows: Section II introduces the terminology and problem formulation. Section III details our generative prediction model. Section IV demonstrates experimental results. Finally, section V concludes this work.

II. PRELIMINARIES

A. Terminology

We define four types of corners, two types of cells, and the pivot value in this work. (see Fig. 2.)

Anchor corner in which full information including the PVT condition, input transition indices, output capacitance indices, and timing values in each timing table is provided. In other words, all timing values in anchor corners are characterized by SPICE simulation. The timing tables in anchor corners serve as the training set.

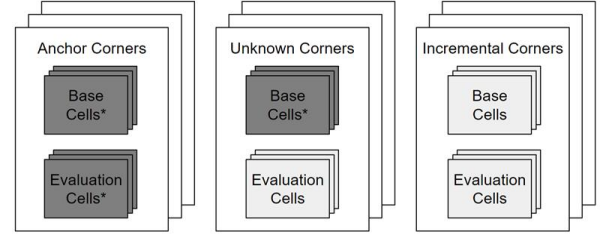


Fig. 2: Base and evaluation cells and the three groups of corners: Cells with a star suffix indicate that their timing values are characterized by SPICE simulation.

Unknown corner in which some of the library cells are characterized by SPICE simulation, and the other cells are characterized by a prediction model. The timing tables in unknown corners serve as the validation set.

Incremental corner in which all library cells are to be characterized by a prediction model. The timing tables in incremental corners are the test set.

Query corner is a corner queried by a user and to be characterized by a prediction model. The corresponding pivot value and timing table indices of a target timing table are encoded as an input to the prediction model. To construct the timing tables for incremental corners, we set each incremental corner as the query corner and use the model to infer and generate its timing tables.

Base cells are library cells which are characterized by SPICE simulation in the unknown corners.

Evaluation cells are library cells which are characterized by a prediction model in the unknown corners.

As shown in Fig. 2, base cells and evaluation cells are characterized by SPICE simulation in anchor corners, but characterized by a prediction model in incremental corners.

Pivot value represents the reference value of a timing table. In our framework, the pivot value is defined as the median value of the table. Taking Table I as an example, the pivot value is 0.04698, highlighted in red.

B. Problem Formulation

As mentioned earlier, accurate and fast cell timing characterization is essential for ensuring design robustness under corner explosion. We attempt to generate incremental cell timing libraries based on SPICE-simulated anchor libraries and corner features. Thus, the problem addressed in this work can be formulated as follows:

The Timing Table Characterization Problem: Given a set of anchor corners with all timing tables of each cell, and a query corner with its PVT configuration, and the table indices and the pivot value of a target timing table, our goal is to characterize the remaining values of the target timing table.

Our goal is to enhance prediction accuracy and reduce the SPICE simulation time. The prediction model is desired to generalize among cells and timing arcs. The timing tables considered in this work include but are not limited to cell rise/fall (propagation) delays and rise/fall transition times.

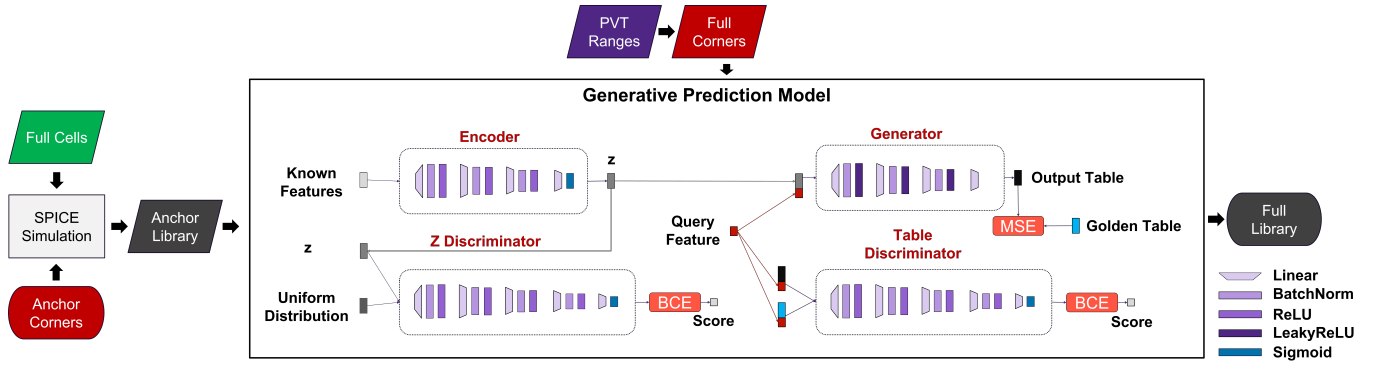


Fig. 3: The overall characterization flow.

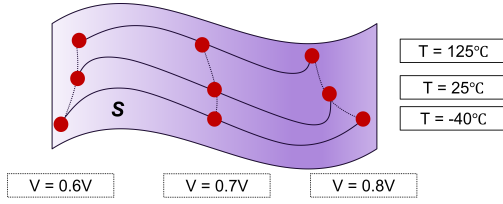


Fig. 4: Cell timing libraries lie on a high-dimensional space S .

III. GENERATIVE PREDICTION MODEL

A. Overview

Fig. 3 demonstrates the overall flow and framework of our cell timing library characterization. Our generative prediction model produces the timing tables for a cell under a query corner, given anchor corners characterized by SPICE simulation. An incremental library, i.e., a cell timing library under an incremental corner, can be generated by querying the full set of cells for such corner.

We can imagine that each timing table is a point lying on a high-dimensional space S as shown in Fig. 4. Exploring the space in the direction of corner changing allows us to uncover its counterparts under various corner settings. We construct a conditional adversarial autoencoder (CAAE) for cell timing library characterization, which uses the pivot value of a cell timing table in the query corner as a conditional input and concatenates it with extracted cell characteristics to generate the whole target timing table.¹

The complete structure of CAAE is shown in Fig. 3. We input the query feature (the pivot point of a cell under the query corner) and known features (extracted from the timing tables at anchor corners), and CAAE will output the whole target timing table. There are four components in CAAE, an encoder, a generator, and two discriminators. The generator and the table discriminator constitute a commonly used GAN structure. The input features of the generator are usually random distributions, which bring a degree of randomness to

the output of the generator. Therefore, we use an encoder to map given cell features into a smooth distribution, then the generator takes the distribution as the input. The randomness no longer exists. The combination of the encoder and generator can also be regarded as an autoencoder. It can be seen that CAAE is a hybrid of GAN and autoencoder. The two discriminators are the z discriminator and table discriminator. The z discriminator is used to guide the encoder to output a uniform distribution. The uniform distribution can help the training process converge faster. The table discriminator is used to grade the output of the generator and to provide feedback for the generator training.

B. Features

The features fed into the generative prediction model are divided into known features and query features. These features are represented as X_{known} and X_{query} , respectively:

$X_{known} = [\text{process, voltage, temperature, output capacitance list, pivot input transition, pivot output capacitance, pivot value, timing values}]$

$X_{query} = [\text{process, voltage, temperature, output capacitance list, pivot input transition, pivot output capacitance, pivot value}]$

The descriptions for each feature are as follows:

- **Process (encoded as 0, 1, 2):** Represents the process type, where 0 corresponds to ff, 1 to tt, and 2 to ss.
- **Voltage:** The operating voltage level.
- **Temperature:** The operating temperature.
- **Output Capacitance List:** Represents different output capacitance values, indexed from the timing table.
- **Pivot Input Transition:** The input transition value corresponding to the pivot point.
- **Pivot Output Capacitance:** The output capacitance value corresponding to the pivot point.
- **Pivot Value:** The median value in a timing table, located at the $\lfloor M/2 \rfloor$ -th row and $\lfloor N/2 \rfloor$ -th column. This value is characterized through SPICE simulation and used to normalize all values in the timing table. Including this value in the features is crucial for ML model training.
- **Timing Values:** All timing values from the timing table. Each table of size $M \times N$ is flattened into a one-dimensional vector, which is then concatenated with other features.

The known feature vector X_{known} includes the complete feature set for the anchor corners, encapsulating all necessary

¹This formulation is analogous to facial age progression and regression, which generates incremental human face images given personal characteristic and age. The CAAE structure is also applied to facial age progression/regression in [11] with a promising image quality. The similarity between the two problems makes CAAE appear suitable for application in timing library characterization.

information from the timing tables. The query feature vector X_{query} represents the PVT configuration for the query corner, along with features related to the pivot value.

C. Input / Output Specifications

Here we specify the input and the output of the encoder, generator, z discriminator, and table discriminator. Let c denote the cell to be characterized by our generative prediction model, z the output of the encoder, and q the query corner.

1) *Encoder*: We first prepare all known features for c in all anchor corners, i.e., if the number of anchor corners is k , we have k known features from X_{known}^1 to X_{known}^k . All known features share the same input transition list because input transition lists are all the same among our benchmark. We concatenate the input transition list and k known features as the input of the encoder. The features can be easily adjusted for variant input transition lists. The output of the encoder is a vector of length 32, and it is expected to follow a uniform distribution.

2) *Generator*: The input of the generator is the concatenation of z and X_{query} of c and q . The output of the generator is a vector of length $M \times N$, which is resized to an $M \times N$ table.

3) *Z Discriminator*: The input of the z discriminator is solely z . The output is a single score. The score increases as the input becomes closer to a uniform distribution.

4) *Table Discriminator*: The input of the table discriminator is the concatenation of a vector of length $M \times N$ and X_{query} . The output is a single score. The score increases as the input is closer to a SPICE-simulated timing table.

D. Model Structures

The internal structures of four components are shown in Fig. 3. They share similar structures. There is a stack of linear layers, batch normalization layers, and activation functions. The difference lies in the selection of the activation function and the Sigmoid function at the end of the encoder, z discriminator, and table discriminator.

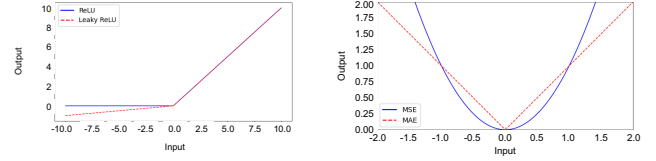
A comparison between ReLU function and LeakyReLU function is shown in Fig. 5a. LeakyReLU function retains information when the input is negative, while ReLU function ignores it. We apply LeakyReLU function on the generator so that the generator will not ignore any information from the already-extracted features. The Sigmoid function is used to confine the output. We expect the output values of the encoder and the two discriminators to range from 0 to 1.

E. Loss Functions

Let n be the length of the predicted and the golden vector, y_i be the i -th element in the golden vector, and $\hat{y}_i$ be the i -th element in the predicted vector. We use two distinct loss functions in our work, mean square error (MSE) and binary cross entropy (BCE). MSE is used in both the encoder and generator, while BCE is used in the two discriminators.

$$MSE(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$BCE(y, \hat{y}) = -\frac{1}{n} \sum_{i=1}^n (\hat{y}_i * \ln y_i + (1 - \hat{y}_i) * \ln (1 - y_i))$$



(a) ReLU and LeakyReLU

(b) MSE and MAE

Fig. 5: Activation and loss functions.

MSE is also known as L2 loss. MSE is the most common loss function for evaluating the distance between the values. MSE tends to provide a more stable result compared to mean absolute error (MAE). MSE is smoother than MAE as shown in Fig. 5b. This is the reason that we apply MSE loss on the comparison of predicted table values and golden table values.

BCE is commonly used in binary classification problems. Discriminators categorize the output into predicted values and golden values. The grading process of the discriminator is also regarded as a binary classification problem. Hence, BCE is adopted for them.

F. Model Training

Due to the competitive nature of the generator and the discriminator, it is challenging to reach Nash equilibrium. The balance between the generator and the discriminator should be carefully maintained. As commonly observed in GANs, we separate the training process into three parts: encoder and generator, z discriminator, and table discriminator.

The encoder and generator are guided not only by the two discriminators but also by the MSE loss function. We measure the distance between the discriminator outputs and the value of 1 through BCE. This makes sense because we want the encoder and generator to deceive the two discriminators, implying that the output should be close to 1. The usage of MSE is similar to that in the typical case. We directly utilize MSE to compare the output of the generator and the ground truth.

For the training process of the z discriminator, we randomly generate a uniform distribution and label it as 1. In contrast, the output of the encoder is labeled as 0. The discriminator should learn to distinguish between them.

For the training process of the table discriminator, the concatenation of the predicted values and X_{query} is labeled as 0, while that of the golden values and X_{query} is labeled as 1. Likewise, the table discriminator should learn to differentiate between the two.

The most difficult part of the training process is to ensure an appropriate balance between the growth of the encoder, generator and discriminators. We employ two techniques to solve this issue. The first one is to backtrack the discriminators once after the encoder and generator have been backtracked m times. This is a common technique used in the training process of GAN. We ensure that the encoder and generator are not overly influenced by the discriminator in the early stage. The second technique is that during the training process of the encoder and generator, we add weights on aforementioned losses. The two loss weights are then adjusted based on the ratio between three types of loss. If the discriminator loss

TABLE II: Corners in two datasets.

Benchmark	Anchor Corners	Unknown Corners	Incremental Corners
CAD Contest	ff0p88vm40c, ff1p1v125c	ff0p99v125c, ff1p1vm40c	ff0p88v125c, ff0p99vm40c
	tt0p9v25c	tt0p8v25c	tt1p0v25c
	ss0p72vm40c, ss0p9v125c	ss0p81v125c, ss0p9vm40c	ss0p72v125c, ss0p81vm40c
SAED14nm	ff0p7vm40c, ff0p88v125c	ff0p7v125c, ff0p88vm40c	ff0p7v25c, ff0p88v25c
	tt0p6vm40c, tt0p8v125c	tt0p6v125c, tt0p8vm40c	tt0p6v25c, tt0p8v25c
	ss0p6vm40c, ss0p72v125c	ss0p6v125c, ss0p72vm40c	ss0p6v25c, ss0p72v25c

is too high, we decrease its weight. It is essential to ensure that the loss from the loss function and the loss from the discriminators are of similar magnitude.

IV. EXPERIMENTAL RESULTS

A. Experimental Settings

Our framework was implemented in the Python 3.9.18 language. The experiments were conducted on a Linux workstation with a 2.1 GHz Intel®Xeon®Gold 6230R CPU equipped with 408 GB RAM, and three NVIDIA GeForce RTX 3090 GPUs equipped with 24 GB RAM on each.

Our experiments were conducted on two datasets: The CAD Contest dataset contains 400 cells and 15 corners [12], while SAED14nm is a 14nm FinFET process library, with 835 cells and 18 corners [13]. Table II provides the corresponding corner lists. The corner name with three segments indicates its nMOS and pMOS processes, voltage, and temperature in order. For the process segment, f, t, and s represent fast, typical, and slow, respectively. For instance, *ss0p9vm40c* corresponds to slow nMOS and pMOS processes, 0.9 V, and -40°C. Moreover, the corners are classified into anchor, unknown, and incremental corners. Given the relative simplicity of interpolation in comparison to extrapolation, the extreme values of voltage and temperature are selected for the anchor corners, encompassing both the highest and lowest observed values.

Cells are divided into two sets: base cells and evaluation cells, with a ratio of 3:7. Each cell has multiple variations with different driving strengths. To avoid excessive duplicates in the train or test set, in our splitting algorithm, cells are grouped based on driving strength into three categories: minimum, median, and maximum. We prioritize selecting base cells from the minimum group first, followed by the maximum group, and then the median group.

B. Results and Comparisons

The overall experimental results of the first experiment are illustrated in Tables III and IV. Linear regression and the FFN proposed in [10] are employed for model comparison. The experimental results are presented in three distinct forms for a comprehensive evaluation of the model performance. The timing values in the standard cell library are directly compared. The three models demonstrate superior performance on transition arcs compared to delay arcs because the relations between indices and values are more straightforward for transition arcs.

Two metrics, MAE and mean absolute percentage error (MAPE), are employed for evaluation. MAPE is the evaluation metric used in 2020 CAD Contest. For SAED14nm, the MAE pass rate is also taken into account [14]. The MAE pass rate threshold is set as 2 ps, which is a commonly accepted industry standard in 14nm process. The pass rates are not

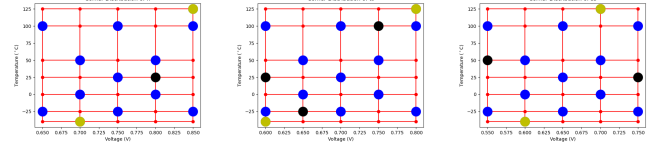


Fig. 6: Anchor corner selection by HDBSCAN. Green points refer to 6 basic anchor corners, black points refer to extra anchor corners, and blue points refer to pre-defined incremental corners.

shown in 2020 CAD Contest results because of the insufficient background information in the benchmark.

MAE is the primary metric used to evaluate model performance, as it directly indicates the absolute error, providing a clear assessment of the model.

C. Model Extensibility

The SAED14nm benchmarks used in the first experiment consist of 18 corners, demonstrating our ability to accurately generate timing tables as compared to the linear regression method and FFN model. To address the corner explosion issue, we conduct the second experiment to evaluate the extensibility and scalability of our model.

Assume there are K corners of interest in total, and we would like to select k anchor corners out of them, where $K \gg k$. First, we devise an anchor corner selection algorithm to pick k anchor corners for training. The algorithm is based on the hierarchical density-based spatial clustering of applications with noise (HDBSCAN), a clustering algorithm. All of the K corners are separated into multiple clusters by HDBSCAN. Then, for each cluster, a fixed proportion of representative corners are selected to represent the cluster. Finally, we use the selected representative corners for the prediction model training. Moreover, we propose a cell selection algorithm to reduce the number of cells to which the corner selection algorithm applies. With the corner selection algorithm, we can scale the number of anchor corners as the total number of corners increases, maintaining a consistent ratio to guarantee performance.

In this experiment, we consider $K = 105$ corners of SAED14nm libraries and utilize different numbers of anchor corners for training. The process covers ff, tt, and ss; voltage levels are 0.6V, 0.65V, 0.7V, 0.75V, and 0.8V; temperatures are -40°C, -25°C, 0°C, 25°C, 50°C, 100°C, 125°C. The golden results characterized by SPICE simulation are generated by Siliconsmart [15] executed in the most basic mode without any acceleration. To ensure a fair comparison, we select 33 corners as incremental corners. The k anchor corners are selected from the remaining 72 corners, and those not selected as anchor corners are used for validation. Fig. 6 illustrates the selected anchor corners and incremental corners. The experimental results are listed in Table V. We demonstrate the results under 6, 12 and 18 anchor corners. (6 means 6 basic anchor corners, while 12 (18) means adding extra 6 (12) anchor corners by the anchor corner selection algorithm.) The model performance improves as the number of corners increases and saturates at 18 anchor corners.

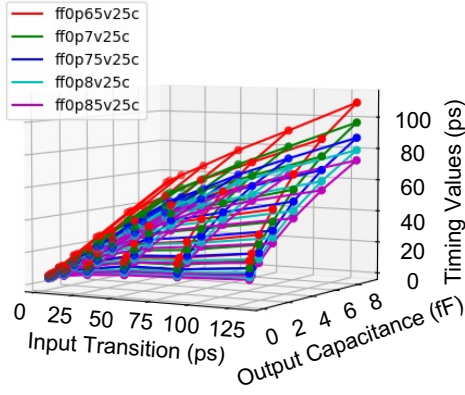


Fig. 7: Cell rise tables of OAI21 at different voltage levels.

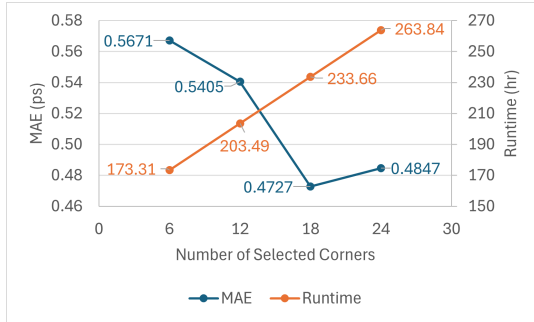


Fig. 8: MAE and runtime with different number of selected anchor corners.

D. Timing Surface Smoothness

As discussed earlier in Section III-A, the generative model effectively captures the high-dimensional space of timing tables across various PVT conditions. To demonstrate smooth traversal and sampling on this high-dimensional surface, we fix the process and temperature while varying the voltage to observe changes in the timing tables. In Fig. 7, we generate cell rise timing tables for the cell OAI21 at different voltage levels, while fixing the process and temperature as ff and 25°C, respectively. The results show continuous and smooth transitions in the timing tables, confirming that our generative model accurately captures the high-dimensional surface, as illustrated in Fig. 4.

E. Runtime and Accuracy Tradeoff

SPICE simulation done by Siliconsmart to characterize the 105 corner libraries takes 302.83 hours. Siliconsmart operates with 32 CPU cores in this process. Our flow's runtime encompasses the time required for characterizing SPICE-simulated data by Siliconsmart, as well as the time taken for cell and corner selection, model training, and model prediction. The model prediction takes only 133.16 seconds to generate all incremental corner libraries. With our flow, SPICE-simulated data can be reduced to one-third of the original volume. The results demonstrate that our characterization flow exhibits 42.8% and 32.8% runtime reduction for 6 and 12 anchor corners compared with Siliconsmart. Combining the model extensibility evaluation with runtime comparison, Fig. 8 depicts the tradeoff between model accuracy and the number of selected anchor corners.

TABLE III: Model accuracy for CAD Contest dataset.

CAD Contest		MAE (ps)			MAPE		
Timing Arc	Process	Linear Regression	FFN Model	Our CAEE	Linear Regression	FFN Model	Our CAEE
Cell Rise	ff	43.8844	8.1800	1.7389	3.72%	6.12%	0.98%
	tt	55.7203	38.3070	2.2125	4.52%	14.53%	1.18%
	ss	65.2358	232.8860	19.8419	5.25%	16.73%	1.66%
Cell Fall	ff	49.9792	6.0266	1.9085	4.38%	4.06%	1.30%
	tt	65.6919	27.6364	2.4238	5.53%	10.87%	1.57%
	ss	75.4952	155.7370	22.5414	6.14%	12.53%	2.00%
Rise Transition	ff	69.6856	5.3936	2.0656	5.42%	5.25%	1.58%
	tt	35.3363	13.7483	2.5037	2.78%	8.28%	1.90%
	ss	72.9652	322.8612	1.3998	7.10%	26.09%	0.81%
Fall Transition	ff	85.9545	3.8278	2.1772	6.85%	4.48%	2.07%
	tt	50.9714	7.9771	3.2441	3.70%	7.21%	2.50%
	ss	98.5066	237.7742	8.3982	9.25%	25.72%	1.40%
Average		64.1188	88.3629	5.8713	5.39%	11.82%	1.58%
Mixed	ff	69.5385	4.8770	2.0969	5.59%	4.35%	1.53%
	tt	50.1529	13.8812	2.9484	4.04%	8.37%	1.79%
	ss	69.9842	886.0233	10.7462	6.33%	38.96%	1.10%
Average		63.2252	301.5398	5.2638	5.32%	17.23%	1.47%

TABLE IV: Model accuracy for SAED14nm dataset.

SAED14nm		MAE (ps)			MAE Pass rate			MAPE		
Timing Arc	Process	Linear Regression	FFN Model	Our CAEE	Linear Regression	FFN Model	Our CAEE	Linear Regression	FFN Model	Our CAEE
Cell Rise	ff	1.3510	1.0813	0.5675	78.26%	86.78%	94.28%	3.19%	2.72%	1.11%
	tt	1.2867	1.2919	0.5675	80.61%	79.58%	95.38%	2.47%	2.34%	0.87%
	ss	1.2662	1.5482	0.8285	80.07%	73.34%	88.73%	1.91%	2.39%	1.06%
Cell Fall	ff	1.3604	1.6923	0.5164	78.69%	67.82%	96.46%	4.23%	4.04%	1.20%
	tt	1.7517	2.0323	0.5105	69.86%	58.46%	97.15%	3.22%	3.89%	0.90%
	ss	1.4735	2.0080	0.5715	75.36%	62.24%	95.76%	2.13%	3.09%	0.91%
Rise Transition	ff	0.4961	0.8048	0.1653	95.95%	91.65%	99.55%	3.55%	9.26%	0.90%
	tt	0.4569	0.8277	0.1881	95.36%	90.71%	99.41%	2.17%	6.50%	0.80%
	ss	0.6111	1.0076	0.2161	94.77%	86.62%	99.27%	2.69%	7.70%	0.86%
Fall Transition	ff	0.6960	1.1460	0.1198	92.03%	81.69%	99.88%	5.20%	6.14%	0.78%
	tt	0.7404	1.0668	0.1525	89.87%	86.11%	99.83%	4.13%	7.21%	0.74%
	ss	0.7547	1.0530	0.2030	90.98%	85.34%	99.76%	3.60%	7.20%	0.78%
Average		1.0204	1.2967	0.3839	85.08%	79.20%	97.12%	3.21%	5.21%	0.90%
Mixed	ff	1.4305	1.3405	0.4214	77.32%	79.63%	96.45%	6.66%	5.64%	1.15%
	tt	1.5802	1.7580	0.5645	74.64%	68.78%	92.97%	6.32%	5.03%	1.12%
	ss	1.6233	1.6113	0.6904	73.91%	71.15%	90.19%	5.25%	4.23%	1.21%
Average		1.5447	1.5699	0.5588	75.29%	73.19%	93.20%	6.08%	4.97%	1.16%

V. CONCLUSIONS

In this work, we propose a cell timing library characterization approach from a generative model perspective. The proposed approach enables designers to generate incremental libraries without the need for confidential data provided by the foundry. The framework not only preserves the adversarial framework of the GAN but also eliminates randomness. A combination of an encoder and a generator is employed to extract salient features from the input, which are then fed into the generator. The model achieves high accuracy with only one-third of the SPICE-simulated data. The model exhibits an exceptional ability to generalize across a wide range of combinational timing arcs. Future work includes extending the approach to constraint timing arcs and aging library characterization.

TABLE V: Model extensibility.

No. of Anchor Corners		MAE (ps)				MAE Pass rate				MAPE			
Timing Arc	Process	6	6+6	6+12	6+18	6	6+6	6+12	6+18	6	6+6	6+12	6+18
Cell Rise	ff	0.4090	0.3338	0.3147	0.3472	98.00%	98.81%	98.91%	98.92%	1.37%	1.12%	1.08%	1.17%
	tt	0.5044	0.4203	0.3985	0.4272	96.94%	97.92%	97.97%	98.07%	1.21%	1.03%	1.00%	1.03%
	ss	0.6892	0.5868	0.5250	0.5573	93.62%	94.92%	96.55%	98.91%	1.19%	1.01%	0.94%	1.00%
Cell Fall	ff	0.4046	0.4453	0.3285	0.3235	97.82%	97.66%	98.57%	98.64%	3.40%	1.72%	1.32%	1.29%
	tt	0.4900	0.5462	0.3977	0.4024	96.81%	96.04%	98.12%	97.88%	1.32%	1.38%	1.05%	1.07%
	ss	0.7660	0.7830	0.5337	0.5788	92.09%	91.97%	96.00%	95.56%	1.38%	1.35%	0.94%	1.04%
Rise Transition	ff	0.4861	0.4361	0.4246	0.4263	97.10%	97.72%	97.75%	97.76%	2.32%	2.08%	2.06%	2.09%
	tt	0.5682	0.5267	0.5272	0.5224	95.68%	96.53%	96.46%	96.56%	2.19%	2.02%	2.03%	2.03%
	ss	0.7620	0.7034	0.6998	0.6933	91.93%	93.10%	93.10%	93.23%	2.29%	2.12%	2.13%	2.09%
Fall Transition	ff	0.4562	0.4711	0.3953	0.3978	96.96%	97.25%	97.89%	97.83%	2.49%	2.63%	2.10%	2.17%
	tt	0.5086	0.5139	0.4701	0.4698	96.47%	96.62%	97.03%	97.10%	2.35%	2.40%	2.06%	2.08%
	ss	0.7410	0.7201	0.6577	0.6705	92.36%	92.81%	93.75%	93.43%	2.46%	2.47%	2.15%	2.17%
Average		0.5671	0.5405	0.4727	0.4847	95.48%	95.95%	96.84%	96.99%	2.00%	1.78%	1.57%	1.60%
Mixed	ff	0.4141	0.3728	0.3546	0.3535	97.64%	98.05%	98.35%	98.48%	1.87%	1.78%	1.63%	1.59%
	tt	0.4901	0.4403	0.4286	0.4249	96.66%	97.38%	97.64%	97.74%	1.65%	1.53%	1.46%	1.44%
	ss	0.7038	0.6020	0.5663	0.5810	93.03%	94.80%	95.38%	95.18%	1.73%	1.55%	1.43%	1.46%
Average		0.5360	0.4717	0.4498	0.4531	95.77%	96.74%	97.13%	97.13%	1.75%	1.62%	1.51%	1.50%

REFERENCES

- [1] Taming corner explosion in complex chips. [Online]. Retrieved February 23, 2023, from <https://semiengineering.com/taming-corner-explosion-in-complex-chips/>.
- [2] Andrew B. Kahng, Uday Mallappa, Lawrence Saul, and Shangyuan Tong. "unobserved corner" prediction: Reducing timing analysis effort for faster design convergence in advanced-node design. In *Proceedings of Design, Automation Test in Europe Conference & Exhibition (DATE)*, pages 168–173, 2019.
- [3] Iris Hui-Ru Jiang. Lightning talk: All routes to timing closure. In *Proceedings of Design Automation Conference (DAC)*, pages 1–2, 2023.
- [4] Haoxing Ren and Matthew Fojtik. Invited-NVCell: Standard cell layout in advanced technology nodes with reinforcement learning. In *Proceedings of ACM/IEEE Design Automation Conference (DAC)*, pages 1291–1294, 2021.
- [5] Aaron C.-W. Liang, Charles H.-P. Wen, and Hsuan-Ming Huang. A general and automatic cell layout generation framework with implicit learning on design rules. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems (TVLSI)*, 30(9):1341–1354, 2022.
- [6] Yih-Lang Li, Shih-Ting Lin, Shinichi Nishizawa, Hong-Yan Su, Ming-Jie Fong, Oscar Chen, and Hidetoshi Onodera. NCTUcell: A DDA- and delay-aware cell library generator for FinFET structure with implicitly adjustable grid map. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 41(12):5568–5581, 2022.
- [7] Tianliang Ma, Zhihui Deng, Xuguang Sun, and Leilai Shao. Fast cell library characterization for design technology co-optimization based on graph neural networks. In *Proceedings of Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 472–477, 2024.
- [8] Xu Cheng, Yuyang Ye, Guoqing He, Qianqian Song, and Peng Cao. Heterogeneous graph attention network based statistical timing library characterization with parasitic RC reduction. In *Proceedings of Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 171–176, 2024.
- [9] Eunice Naswali, Namhoon Kim, and Pravin Chandran. Fast and accurate library generation leveraging deep learning for OCV modelling. In *Proceedings of International Symposium on Quality Electronic Design (ISQED)*, pages 349–354, 2021.
- [10] Seyed Milad Ebrahimipour, Behnam Ghavami, Hamid Mousavi, Mohsen Raji, Zhenman Fang, and Lesley Shannon. Aadam: A fast, accurate, and versatile aging-aware cell library delay model using feed-forward neural network. In *Proceedings of International Conference on Computer Aided Design (ICCAD)*, pages 1–9, 2020.
- [11] Zhifei Zhang, Yang Song, and Hairong Qi. Age progression/regression by conditional adversarial autoencoder. In *Proceedings of Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4352–4360, 2017.
- [12] 2020 CAD Contest in fast timing model estimation for new pvt. [Online]. Retrieved July 18, 2023, from https://drive.google.com/file/d/1XDLVPTlumdFpFPv8TnkKE5wIWrd_EYy-/view.
- [13] Synopsys, Inc. University software program: SAED_EDK14_FINFET. Retrieved July 18, 2023, from <https://www.synopsys.com/academic-research/university.html>.
- [14] Megan Marsh and Wei-Lii Tan. Improving library characterization quality and runtime with machine learning. [Online]. Retrieved July 18, 2023, from <https://resources.sw.siemens.com/en-US/white-paper-improving-library-characterization-with-machine-learning>.
- [15] Synopsys, Inc. Siliconsmart, version 2020.12. Retrieved December, 2020, from <https://www.synopsys.com/academic-research/university.html>.